

3.5 Advanced Conditionals

Sue is working on a program that has a simple menu-based user interface. The main menu has six options, each one associated with a letter on the keyboard. She could create a large IF/ELSE statement to implement this menu, but that solution seems tedious and inelegant. Fortunately she has just learned about SWITCH statements which seem like the perfect tool for the job.

Objectives

By the end of this class, you will be able to:

- Create a SWITCH statement to modify program flow.
- Create a conditional expression to select between two expressions.

Prerequisites

Before reading this section, please make sure you are able to:

- Declare a Boolean variable (Chapter 1.5).
- Convert a logic problem into a Boolean expression (Chapter 1.5).
- Recite the order of operations (Chapter 1.5).
- Create an IF statement to modify program flow (Chapter 1.6).
- Recognize the pitfalls associated with IF statements (Chapter 1.6).

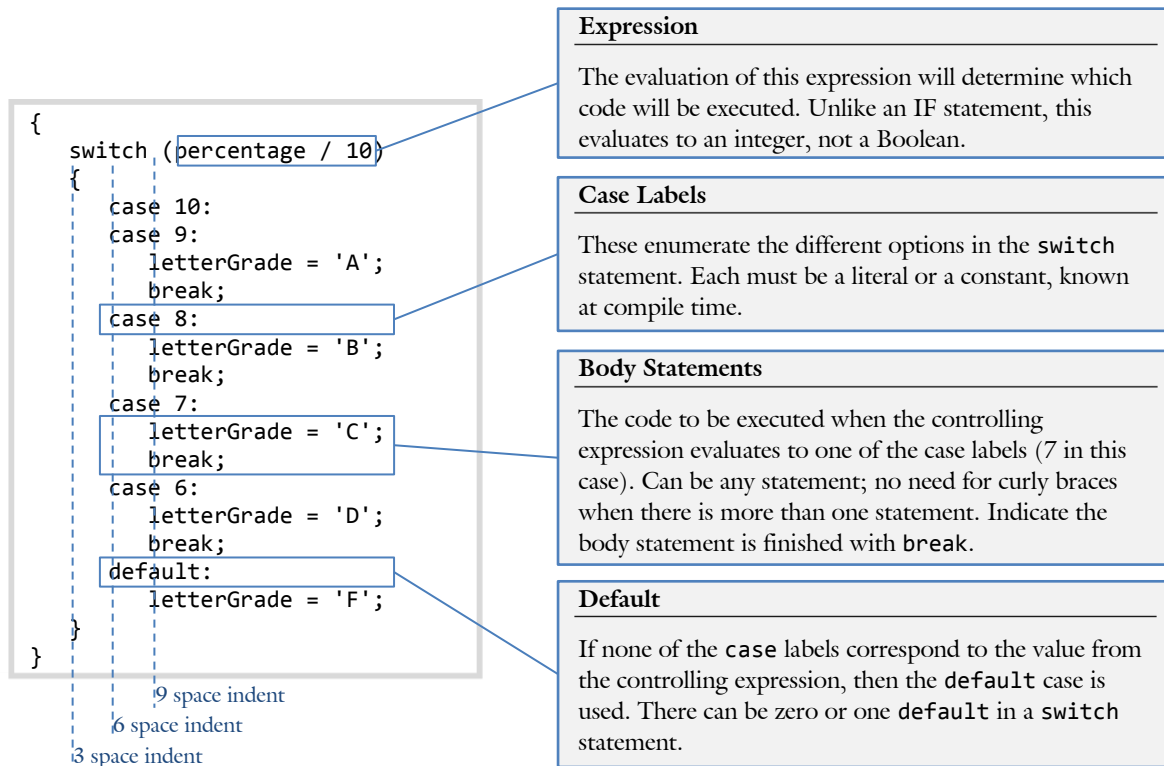
Overview

A fundamental component of any programming language is decision-making logic. Up to this point, the only decision-making mechanism we have learned is the IF statement. While this is a useful and powerful construct, it is actually rather limited: it will only help you choose between two options. We have worked around this restriction by nesting IF statements (Chapter 1.6) and using arrays (Chapter 3.0). This chapter will introduce three new decision making tools: SWITCH statements, conditional expressions, and bitwise operators.

Switch	Conditional Expression	Bitwise Operators
Allows the programmer to choose between more than two options. Each option is specified with an integral value. <pre>switch(option) { case 'Q': return true; case 'D': display(data); break; ... }</pre>	Useful for selecting between two expressions or values (rather than two statements as an IF statement does). <pre>cout << "Greetings " << (isMale ? "Mr." : "Mrs.") << lastName << endl;</pre>	Enables a single integer to store 32 Boolean values worth of data for a more compact representation of an array of bools. <pre>int value = 0x0001 // 1st bit 0x0004 // 3rd bit 0x0200; // 10th bit if (value & 0x001) cout << "1st bit";</pre>

Switch

Most decisions in programming languages are made with IF statements. This works great if there are only two possible decisions. While it is possible to use multiple IF statements to achieve more than two possibilities (think of the tax function from Project 1), the answer is less than elegant at times. The SWITCH statement provides the ability to specify more than two possibilities in an elegant and efficient way.



Expression

The SWITCH statement is driven by a controlling expression. This expression will determine which CASE will be executed. The best way to think of the controlling expression is like a switch operator for a railroad track. There are a finite number of tracks down which the operator can send a train, each identified by a track number. The operator's job is to match the physical track with the requested track number. Similarly, the controlling expression in a SWITCH statement determines which CASE label to execute.

The controlling expression must evaluate to an integer. Evaluation to a `bool`, `char`, or `long` works as well because each can be readily converted to an `int`. You cannot use a floating point number or pointer as the data-type of the controlling expression.

For example, consider the following code to implement a user interface menu. The following menu options are presented to the user:

```

Please enter an option:
  S ... Save the game
  N ... New game
  P ... Play the current game
  D ... Display the current status
  Q ... Quit
  ? ... Display these options again
  
```

The following code will implement the menu:

```
/* *****
 * EXECUTE COMMAND
 * Execute command as specified
 * by the caller
 * ***** */
void executeCommand(char option) // note that this is a char, not an integer
{
    switch (option)                // because a char readily converts to an integer, this
    {                             // is not a problem
        case 'S':                 // 'S' corresponds to 83 on the ASCII table
            saveGame(game);        // execute the function corresponding to the 'S' input
            break;                 // finished with this case
        case 'N':
            newGame(game);
            break;
        ...                       // there are more options here of course
    }                             // don't forget the closing curly brace
}
```

The controlling expression can do more complex arithmetic. This is commonly the case when working with floating point numbers. In this second example, we are converting a GPA into a more workable medium:

```
/* *****
 * DISPLAY GRADE
 * Display the letter grade version
 * of a GPA
 * ***** */
void displayGrade(float gpa) // value is originally a float
{
    switch ((int)(gpa * 10.0)) // note how we cast to an integer
    {
        case 40:                 // corresponding to a 4.0 GPA
            cout << "A+";
            break;
        case 39:                 // corresponding to a 3.9 --> 3.3 GPA
        case 38:
        case 37:
        case 36:
        case 35:
        case 34:
        case 33:
            cout << "A";
            break;
        ...
    }
}
```

When the controlling expression cannot be converted to an integer, then we will get a compile error:

```
{
    char name[] = "CS 124";
    switch (name)                // ERROR! Pointers cannot be converted to an integer
    {
        ...
    }
}
```

The problem here is that we cannot readily convert a pointer into an integer. Typically some non-trivial processing needs to happen before this can be done. Of course, the programmer could just cast the address into an integer, but that would probably be a bug!

Floating point numbers also cannot be used for the controlling expression. This makes sense when you realize that floating points are approximations.

```
{
    float pi = 3.14159;
    switch (pi)                // ERROR! Floating point numbers cannot be converted
    {                          //      to an integer without casting
        ...
    }
}
```

Case labels

Back to our train-track analogy, the CASE label in a SWITCH statement corresponds to a track number. Note that, aside from Harry Potter's wizarding world, there is no such thing as platform 9³/₄. Similarly, each CASE label must be an integer.

There is an additional constraint. In the C++ language, the compiler must know at compile time each CASE label. This means that the value must be a constant (`const int VALUE = 4;`) or a literal (4), it cannot be a variable. The final constraint is that each label must be unique. Imagine the confusion of the train operator trying to determine *which* "track 12" the train wants!

The first example corresponds to standard integer literal case numbers. In this case, we are converting a class number into a class name:

```
/******
 * DISPLAY CLASS NAME
 * Convert a class number (124) into a
 * name (Introduction to Software Development)
 *****/
void displayClassName(int classNumber)
{
    switch (classNumber)          // classNumber must be an integer
    {
        case 124:
            cout << "Introduction to Software Development\n";
            break;
        case 165:
            cout << "Object Oriented Software Development\n";
            break;
        case 235:
            cout << "Data Structures\n";
            break;
        case 246:
            cout << "Software Design & Development\n";
            break;
    }
}
```

In the second example, we have a character literal as the case label. In this case, we will be displaying the name for a typed letter:

```

/*****
 * DISPLAY LETTER NAME
 * Display the full name ("one") corresponding
 * to a given letter ('1')
 *****/
void displayLetterName(char letter)
{
    switch (letter)                // though letter is a char, it readily converts
    {                             // to an integer
        case 'A':                 // character literal, corresponding to 65
            cout << "Letter A";
            break;
        case '1':                 // character literal 48
            cout << "Number one";
            break;
        case 32:                  // corresponding to the character literal ' '
            cout << "Space";
            break;
    }
}

```

Finally, we can use a constant for a case label. Here, the compiler guarantees the value cannot be changed.

```

const int GOOD  1;
const int BETTER 2;
const int BEST  3;

/*****
 * DISPLAY
 * convert a value into the name
 *****/
void display(int value)
{
    switch (value)
    {
        case BEST:
            cout << "Best!\n";
            break;
        case BETTER:
            cout << "Better!\n";
            break;
        case GOOD:
            cout << "Good!\n";
            break;
    }
}

```

There are three common sources of errors with case labels. The first is to use something other than an integer. This will never correspond to the integer resulting from the controlling expression. The second is to use a variable rather than a literal or constant. The final is to duplicate a case label:

```

{
    int x = 3;
    switch (4)
    {
        case "CS 124":      // ERROR! Must be an integer and this is a pointer to a char
            break;
        case 3.14159:      // ERROR! This is a float and needs to be an integer
            break;

        case x:            // ERROR! This is a variable and it needs to be a literal
            break;

        case 2:
            break;
        case 2:            // ERROR! Duplicate value
            break;
    }
}

```



Sue's Tips

It turns out that a SWITCH statement is much more efficient than a collection of IF/ELSE statements. The reason for this has to do with how compilers treat SWITCH statements. Typically, the compiler creates something called a “perfect hash” which allows the program to jump to exactly the right spot every time. Thus, a SWITCH with 100 CASE labels is just as efficient as one with only 3. The same cannot be said for IF/ELSE statements!

Default

The DEFAULT keyword is a special CASE label corresponding to “everything else.” In other words, it is the catch-all. If none of the other CASE labels correspond to the result of the controlling expression, then the DEFAULT is used. There can be either zero or one default in a switch statement. Typically we put the default at the end of the list but it could be anywhere.

The first example illustrates using a DEFAULT where multiple CASEs would otherwise be needed:

```

switch (numberGrade / 10)
{
    case 10:
    case 9:
        cout << "Perfect job";
        break;
    case '8':
    case '7':
        cout << "You passed!\n";
        break;
    default:                // covering 6, 5, 4, 3, 2, 1, and 0
        cout << "Take the class again\n";
}

```

In these cases, the DEFAULT is used to minimize the size of the source code and to increase efficiency. It is useful to put a comment describing what cases the DEFAULT corresponds to.

The second example uses the DEFAULT to handle unexpected input. In these cases, there is typically a small number of acceptable input values and a very large set of invalid input values:

```

{
    char input;
    cout << "Do you want to save your file (y/n)? ";
    cin >> input;                // though a char is accepted, there are only
                                // two valid inputs: 'y' and 'n'
    switch (input)                // 256 possible values, but only 2 matter
    {
        case 'Y':                // we are treating 'Y' and 'y' the same here
        case 'y':
            save(data);
            break;
        case 'n':                // the "do-nothing" condition
        case 'N':
            break;
        default:                 // everything else
            cout << "Invalid input '"
                 << input
                 << "'. Try again\n";
    }
}

```

The most common error with DEFAULT statements is to try to define more than one.

Body statements

You can put as many statements inside the CASEs as you choose. The {}s are only needed if you are declaring a variable. To leave a switch statement, use the BREAK statement. This will send execution outside the SWITCH statement. Note that the BREAK statement is optional.

```

/*****
 * EXECUTE COMMAND
 * Execute command as specified
 * by the caller
 *****/
void executeCommand(char option)
{
    switch (option)
    {
        case 's':                // When there are two case labels like this, the
                                // fall-through is implied
        case 'S':                // Save and Quit
            saveGame(game);
            // fall-through... We don't want a break because we want the 'Q' condition next
        case 'q':
        case 'Q':                // Quit
            quit(game);
            break;
    }
}

```

A common error is to forget the BREAK statement between cases which yields an unintentional fall-through. If a fall-through is needed, put a comment to that effect in the code.

Example 3.5 – Golf

Demo

This example will demonstrate a simple SWITCH statement to enable the program to select between six different options.

Problem

In the game of golf, each hole has a difficulty expressed in terms of how many strokes it takes for a standard (read “a very good”) golfer to complete. This standard is called “par.” If a golfer completes a hole in one fewer strokes than par, he is said to achieve a “birdie” (no actual birds are used in this process). If he does two better, he achieves an “eagle.” Finally, if he takes one more stroke than necessary, he gets a “bogie.” Write a function to convert a score into the corresponding label.

```
What is your golf score? 3
What is par for the hole? 5
You got an eagle
```

Solution

The important work is done in the following function:

```
/******
 * DISPLAY
 * Translate the golfer performance into
 * a "bogie," "par," or whatever
 *****/
void display(int score, int par)
{
    // translate the golfer performance into a "bogie", or "par" or whatever
    switch (score - par)
    {
        case 2:
            cout << "You got a double bogie\n";
            break;
        case 1:
            cout << "You got a bogie\n";
            break;

        case 0:
            cout << "You got par for the hole\n";
            break;

        case -1:
            cout << "You got a birdie\n";
            break;
        case -2:
            cout << "You got an eagle\n";
            break;

        default:
            cout << "Your score was quite unusual\n";
    }

    return;
}
```

See Also

The complete solution is available at [3-5-golf.cpp](#) or:

```
/home/cs124/examples/3-5-golf.cpp
```



Conditional Expression

While an IF statement chooses between two *statements*, a conditional expression chooses between two *expressions*. For example, consider the following code inserting a person's title before their last name according to their gender:

```
cout << "Hello "  
      << (isMale ? "Mr. " : "Mrs. ")  
      << lastName;
```

Even though we have a single cout statement, we can embed the conditional expression right in the middle of the statement.

Up to this point, we have used **unary operators** (operators with a single operand) such as increment (++a), logical not (!a), address-of (&a) and dereference (*a). We have also used **binary operators** (operators with two operands) such as addition (a + b), modulus (a % b), logical and (a && b), greater than (a > b), and assignment (a += b). There is exactly one **ternary operator** (operator with three operands) in the C++ language: the conditional expression:

```
<Boolean expression> ? <true expression> : <false expression>
```

Like all operators, the result is an expression. In other words, the evaluation of the conditional expression is either the true-expression or the false-expression.

Note that the conditional expression operator resides midway on the order of operations table. Because the insertion (<<) and the extraction (>>) operator are above the conditional expression in the order of operations, we commonly need to put parentheses around conditional expressions.

Example 1: Absolute value

Consider, for example, the absolute value function. In this case, we return the value if the value is already positive. Otherwise, we return the negative of the value. In other words, we apply the negative operator only if the value is already negative.

```
number = (number < 0) ? -number : number;
```

Of course this could be done with an IF statement, but the conditional expression version is more elegant.

Example 2: Minimum value

In another example, we would like to find the smaller of two numbers:

```
lower = (number1 > number2) ? number2 : number1;
```

Here we are choosing between two values. The smaller of the two will be the result of the conditional expression evaluation and subsequently assigned to the variable lower.



Sue's Tips

Conditional expressions have comparable performance characteristics to IF statements; compilers typically treat them in a similar way. Some programmers avoid conditional expressions because they claim it makes the code more difficult to read. Some programmers favor conditional expressions because they tend to make the code more compact. In general, this is a stylistic decision. As with all stylistic issues, favor a design that increases code clarity and exposes potential bugs.

Example 3.5 – Select Tabs or Newlines

Demo

This example will demonstrate how to use conditional expressions to choose between options. While an IF statement could do the job, conditional expressions are more elegant.

Problem

Write a program to display the multiplication tables between 1 and n. We wish to put a tab between each column but a newline at the end of the row. In other words, we put a tab after every number except the number at the end of the row:

```
How big should your multiplication table be? 4
1      2      3      4
2      4      6      8
3      6      9     12
4      8     12     16
```

Solution

To display two-dimensional data such as a multiplication table, it is necessary to have two counters: one for the row and one for the column. After each number in the table, we will have either a newline '\n' or a tab '\t'. We choose which is to be used based on the column number.

```
void displayTable(int num)
{
    for (int row = 1; row <= num; row++)           // count through the rows
        for (int col = 1; col <= num; col++)        // count through the columns
            cout << (row * col)                    // display the product
                << (col == num ? '\n' : '\t');      // tab or newline, depending
}
```

Notice how the function could have been done with an IF statement, but it would not have been as elegant:

```
void displayTable(int num)
{
    for (int row = 1; row <= num; row++)           // count through the rows
        for (int col = 1; col <= num; col++)        // count through the columns
        {
            if (col == num)                          // are we on the last column?
                cout << (row * col) << endl;        // display an endl
            else
                cout << (row * col) << '\t';        // display a tab
        }
}
```

Challenge

As a challenge, modify the above function to handle negative values in the num parameter. If a negative value is passed, make it a positive value using absolute value. Implement absolute value using a conditional expression.

See Also

The complete solution is available at [3-5-selectTabOrNewLine.cpp](#) or:

```
/home/cs124/examples/3-5-selectTabOrNewline.cpp
```



Bitwise Operators

To understand bitwise operators, it is first necessary to understand a bit. As you may recall from Chapter 1.2, data is stored in memory through collections of bits. There are 8 bits in a byte and an integer consists of 4 bytes (32 bits). With computers, we represent numbers in binary (base 2 where the only possible values are 0 and 1), decimal (base 10 where the only possible values are 0...9), and hexadecimal (base 16 where the possible values are 0...9, A...F). Consider, for example, the binary value 00101010. The right-most bit corresponds to 2^0 , the next corresponds to 2^1 , and the next corresponds to 2^2 . Thus 00101010 is:

$$\begin{array}{cccccccc} 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ \hline 0 & + & 0 & + & 32 & + & 0 & + & 8 & + & 0 & + & 2 & + & 0 & = & 42 \end{array}$$

In other words, each place has a value corresponding to it (as a power of two because we are counting in binary). You add that value to the sum only if there is a 1 in that place. This is how we convert the binary 00101010 in the decimal 42. Hexadecimal is similar to decimal except we are storing a nibble (4 bits for 2^4 possible values) into a single digit. Thus there are 3 ways to count to 16, the binary way, the decimal way, and the hexadecimal way:

Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	10000
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Hexadecimal	0x00	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D	0x0E	0x0F	0x10

In many ways, a byte is an array of eight bits. Since each bit stores a `true/false` value (similar to a `bool`), we should be able to store eight Boolean values in a single byte. The problem, however, is that computers find it easier to work with bytes rather than bits (hence `sizeof(bool) == 1`). Wouldn't it be great if we could access individual bits in a byte? We can with bitwise operators.

Bitwise operators allow us to perform Boolean algebra not on Boolean variables but on individual bits. We have six bitwise operators:

Operator	Description	Example
<code>~</code>	Bitwise NOT	<code>0101 == ~1010</code>
<code>&</code>	Bitwise AND	<code>1000 == 1100 & 1010</code>
<code> </code>	Bitwise OR	<code>1110 == 1100 1010</code>
<code>^</code>	Bitwise XOR	<code>0110 == 1100 ^ 1010</code>
<code><<</code>	Left shift	<code>0110 == 0011 << 1</code>
<code>>></code>	Right shift	<code>0110 == 1100 >> 1</code>

One common use of bitwise operators is to collapse a collection of Boolean values into a single integer. If, for example, we have a variable called `settings` containing these values, then we can turn on bits in `settings` with the bitwise OR operator `|`. We can then determine if a setting is on with the bitwise AND operator `&`.

Consider, for example, the following daily tasks Sue may need to do during her morning routine:

```
#define takeShower    0x01
#define eatBreakfast  0x02
#define getDressed   0x04
#define driveToSchool 0x08    // observe how each literal refers to a single bit
#define driveToChurch 0x10
#define goToClass     0x20
#define doHomework    0x40
#define goOnHike      0x80
```

Observe how each value corresponds to turning on a single bit. We can next identify common tasks:

```
#define weekDayRoutine = takeShower | eatBreakfast |
                        getDressed | driveToSchool |
                        doHomework                // use the bitwise OR
#define saturdayRoutine = eatBreakfast | getDressed | // to combine settings.
                        goOnHike                // This will set many
#define sundayRoutine   = takeShower | eatBreakfast | // individual bits
                        getDressed | driveChurch
```

With the bitwise OR operator, we are adding individual bits to the resulting value. Now when the code attempts to perform these tasks, we use the bitwise AND to see if the bits are set:

```
{
    unsigned char setting = sundayRoutine;

    // take a shower?
    if (setting & takeShower)                // use the bitwise AND to check
        goTakeAShower();                    // if an individual bit
                                            // is set. Be careful to
                                            // not use the && here... it
    // eat breakfast?                        // will always evaluate
    if (setting & eatBreakfast)              // to TRUE
        goEatBreakfast();

    // get dressed?
    if (setting & getDressed)
        goGetDressed();
}
```

Bitwise operators are rarely used in typical programming scenarios. They can be seen in system programming where programs are talking to hardware devices that use individual bits for control and status reporting. However, when you encounter them, you should be familiar with what they do.

Sam's Corner



It turns out that we could have been using bitwise operators since the very beginning of the semester. Remember how we format floating point numbers for output:

```
cout.setf(ios::fixed);
cout.setf(ios::showpoint);
cout.precision(2);
```

It turns out that the `setf()` method of `cout` uses bitwise operators to set configuration data:

```
cout.setf(ios::fixed | ios::showpoint);
cout.precision(2);
```

Since `ios::fixed == 4 (22)` and `ios::showpoint == 1024 (210)`, we could also be truly cryptic and say:

```
cout.setf((std::ios_base::fmtflags)1028); // need to cast it to fmtflags
cout.precision(2);
```

Example 3.5 – Show Bits

Demo

This example will demonstrate how to look at the individual bits of a variable. This will be accomplished by looping through all 32 bits of an integer, masking away each individual bit with the bitwise and & operator.

Problem

Write a program to display the bits of a number, one at a time.

```
Please enter a positive integer: 42  
The bits are: 0000000000000000000000000000101010
```

Solution

The first step to solving this problem is to create a number (called `mask`) with a single bit in the leftmost place. When this mask is bitwise ANDed against the target number (`mask & value`), the resulting expression will evaluate to `true` only if there is a 1 in that place in the target number. Next, the 1 in the mask is shifted to the right by one space (`mask = mask >> 1`) and the process is repeated.

```

/*****
 * DISPLAY BITS
 * Display the bits corresponding to an integer
 *****/
void displayBits(unsigned int value)
{
    unsigned int mask = 0x80000000;           // only the left-most bit is set

    for (int bit = 31; bit >= 0; bit--)        // go through all 32 bits
    {
        cout << ((mask & value) ? '1' : '0'); // check the current bit
        mask = mask >> 1;                     // shift the mask by one space
    }
    cout << endl;
}

```

Challenge

As a challenge, make a char version of the above function. How big must the mask be? How many bits will it have to loop through?

See Also

The complete solution is available at [3-5-showBits.cpp](#) or:

```
/home/cs124/examples/3-5-showBits.cpp
```



Problem 1

What is the output of the following code?

```
{
    int a = 0;
    int b = 1;
    int * c = &b;
    *c = 2;
    int * d = &a;
    b = 3;
    d = &b;
    *d = 4;
    cout << "b == " << b << endl;
}
```

a	b	c	d

Answer:

Please see page 258 for a hint.

Problem 2

What is the output of the following code?

```
{
    char a[] = "Banana";
    char b[] = "Bread";
    char * c;

    if (a == b)
        c = a;
    else
        c = b;

    cout << c << endl;
}
```

Answer:

Please see page 277 for a hint.

Problem 3

What is the output of the following code?

```
{
    int number = 5;

    switch (number)
    {
        case 4:
            cout << "four!\n";
            break;
        case 5:
            cout << "five!\n";
            break;
        case 6:
            cout << "six!\n";
            break;
        default:
            cout << "unknown!\n";
    }
}
```

Answer:

Please see page 285 for a hint.

Problem 4

What is the syntax error in the following code?

```
{
    int input = 20;

    switch (input)
    {
        case 'a':
            cout << "A!\n";
            break;
        case true:
            cout << "B!\n";
            break;
        case 2.0:
            cout << "C!\n";
            break;
        default:
            cout << "unknown!\n";
    }
}
```

Answer:

Please see page 287 for a hint.

Problem 5

What is the output of the following code?

```
{
    float grade1 = 3.7;
    int    grade2 = 60;

    switch ((int)(grade1 * 2.0))
    {
        case 8:
            grade2 += 5;
        case 7:
            grade2 += 5;
        case 6:
            grade2 += 5;
        case 5:
            grade2 += 5;
        case 4:
            grade2 += 5;
        default:
            grade2 += 10;
    }

    cout << grade2 << endl;
}
```

Answer:

Please see page 290 for a hint.

Problem 6

Write a function that take a letter as input and displays a message on the screen:

Letter	Message
A	Great job!
B	Good work!
C	You finished!
All other	Better luck next time

Answer:

Please see page 285 for a hint.

Assignment 3.5

The purpose of this assignment is to demonstrate `switch` statements and conditional operators. Though of course it is possible to complete this assignment without using either, it will defeat its purpose.

Your assignment is to write two functions (`computeLetterGrade()` and `computeGradeSign()`) and a single driver program to test them.

computeLetterGrade

Write a function to return the letter grade from a number grade. The input will be an integer, the number grade. The output will be a character, the letter grade. You must use a `switch` statement for this function. Please see the syllabus for the meaning behind the various letter grades.

computeGradeSign

Write another function to return the grade sign (+ or -) from a number grade. The input will be the same as with `computeLetterGrade()` and the output will be a character. If there is no grade sign for a number grade like 85%=B, then return the symbol '*'. **You must use at least one conditional expression.** Please see the syllabus for the exact rules for applying the grade sign.

Driver Program

Create a `main()` that prompts the user for a number graded then displays the letter grade.

Example

Three examples... The user input is underlined.

Example 1: 81%

```
Enter number grade: 81
81% is B-
```

Example 2: 97%

```
Enter number grade: 97
97% is A
```

Example 3: 77%

```
Enter number grade: 77
77% is C+
```

Assignment

The test bed is available at:

```
testBed cs124/assign35 assignment35.cpp
```

Don't forget to submit your assignment with the name "Assignment 35" in the header.

Please see page 49 for a hint.